

hw05

Please make sure you're following all the instructions described in the "Homework Instructions" page on Canvas (go to the homepage of our class and you'll see a link saying "Homework Instructions" just underneath the header of the page).

NOTE: Remember that you may need to use the `t()` function to transpose the result from `apply()` so that it matches the desired output (which will be specified in the the question). Also note that in some cases you'll need to add row and/or column names to the output of `apply()` to match the desired output.

1 Row and column means

For this exercise we'll be using the built-in dataset `JohnsonJohnson`, which contains the quarterly earnings per Johnson & Johnson share from 1960 to 1980.

The data comes as a time series (`ts`) object - however, we can convert it to a ordinary numeric vector like so:

```
jj <- as.numeric(JohnsonJohnson)
```

1.1 *Create a matrix*

First, create a matrix containing the data. Each row should represent a row and each column should represent a quarter (so there should be four columns). (The first row should be 0.71 0.63 0.85 0.44.)

The columns should be named “Q1”, “Q2”, “Q3”, and “Q4”. The row names should be the year (1960 through 1980).

var: `jj_mat`

1.2 *Yearly means*

Use `apply` to find the mean for each year.

var: `jj_mn_yr`

1.3 *Quarterly means*

Use `apply` to find the mean for each quarter.

var: `jj_mn_qtr`

1.4 *Mean and SD*

2 League standings

Suppose that there is an intramural soccer league with 4 teams that has been running for 6 years. The following code creates a matrix with the points for each team in each year.

```
points <- rbind(
  c(13, 10, 6, 16),
  c(17, 5, 9, 13),
  c(12, 8, 15, 4),
  c(11, 7, 14, 18),
  c(9, 5, 16, 13),
  c(8, 12, 15, 19)
)
colnames(points) <- paste0("Team ", LETTERS[1:ncol(points)])
rownames(points) <- paste0("Year ", 1:nrow(points))
```

{Note: AI was used to generate a realistic matrix of points}

2.1 Order the team names

The value for each cell represents that team's points for that season. Let's see if we can create a matrix of the team names where the first column contains the name of the first-place team, the second column contains the name of the second-place team, and so on. The final result should look like this:

	First	Second	Third	Fourth
Year 1	"Team C"	"Team B"	"Team A"	"Team D"
Year 2	"Team B"	"Team C"	"Team D"	"Team A"
Year 3	"Team D"	"Team B"	"Team A"	"Team C"
Year 4	"Team B"	"Team A"	"Team C"	"Team D"
Year 5	"Team B"	"Team A"	"Team D"	"Team C"
Year 6	"Team A"	"Team B"	"Team C"	"Team D"

Let's break this into steps.

2.1.1 Step 1

First, we need to be able to order the names according to their point value. Let's consider how to do this for a single vector of points.

```
single_row <- points[1, ] # get the first row
single_row
```

```
Team A Team B Team C Team D
    13    10     6    16
```

The `order()` function is a handy function for ordering vectors. It returns the indices of the elements in order. Let's use an example to see what it does:

```
x <- c(98, 75, 85, 92)
order(x)
```

```
[1] 2 3 4 1
```

The ordering 2, 4, 3, 1 indicates to use that the order of the elements is, from smallest to largest:

1. Element 2 (1)
2. Element 4 (3)
3. Element 3 (5)
4. Element 1 (8)

We can use this to order vectors. For example, we can use this to sort `x` itself:

```
x[order(x)]
```

```
[1] 75 85 92 98
```

But we can also use it to sort *other* vectors. For example, suppose `x` represented test scores, and `names` represents the name of the student. We can use `order()` to order the names by their test score from highest to lowest:

```
names <- c("Bob", "Jane", "Bill", "Mary")
names[order(x, decreasing = TRUE)]
```

```
[1] "Bob" "Mary" "Bill" "Jane"
```

Note that `decreasing = TRUE` sorts in descending order (highest to lowest).

Use `order()` with `single_row` to return a character vector of the team names, ordered by the number of points. The result should be:

```
var: single_ordered
```

2.1.2 Step 2

Now use the code you just wrote to create a function that does what you just did for any vector you pass to it (returns the names of a numeric vector ordered by the values).

var: `order_names` - should accept one parameter (the vector) and output the names ordered by the values

2.1.3 Step 3

Now use the function in combination with `apply` to perform this operation for all the rows in the matrix. Make sure that each row represents a year (you may need to transpose the matrix). Name the columns "First", "Second", "Third", and "Fourth".

```
var: teams_ordered
```

2.2 *Getting the rankings*

Rather than the points they earned in that season, let's create a matrix that shows each team's rank. The result should look like this:

	Team A	Team B	Team C	Team D
Year 1	2	3	4	1
Year 2	1	4	3	2
Year 3	2	3	1	4
Year 4	3	4	2	1
Year 5	3	4	1	2
Year 6	4	3	2	1

The `rank()` function could be useful here. Note that it only ranks things in order from smallest to largest (so the team with the lowest point total will be assigned 1). How could we transform the matrix to make it rank the values in descending order?

```
var: ranks
```

2.3 *Best year*

Use `apply` to find the year in which each team had the most points as well as the number of points they had that year. The final result should look like this:

	Team A	Team B	Team C	Team D
Points	17	12	16	19
Year	2	6	5	6

You'll need to give the names to match the names above.

Hint: the `which.max()` function will be useful here.

```
var: best_years
```

3 Running times

Suppose that we have the following matrix of race times in minutes for four runners across five 5Ks:

```
race_times <- rbind(
  c(23.4, 22.1, 22.3, 24.1, 23.1),
  c(18.4, 18.1, 18.5, 19.0, 18.2),
  c(27.8, 28.1, 27.1, 26.9, 27.8),
  c(16.7, 16.4, 16.5, 16.9, 16.6)
)
rownames(race_times) <- c("George", "John", "Thomas", "James")
colnames(race_times) <- paste0("Race ", LETTERS[1:ncol(race_times)])
race_times
```

	Race A	Race B	Race C	Race D	Race E
George	23.4	22.1	22.3	24.1	23.1
John	18.4	18.1	18.5	19.0	18.2
Thomas	27.8	28.1	27.1	26.9	27.8
James	16.7	16.4	16.5	16.9	16.6

3.1 *Difference from the mean*

Create a new matrix where each cell contains the result of subtracting the runner's average race time from the time for that specific race. The result should look like this:

```
var: race_mean_diff
```

3.2 *Change in time per race*

For each runner, calculate the change in time for one race to the next. Note that the resulting matrix should have only four columns, not five - it should look like this:

If you don't remember the function for calculating the first difference of a vector, Google it.

```
var: race_diffs
```

3.3 *Fastest time*

Retrieve each runner's fastest time.

```
var: fastest_times
```

3.4 *Fastest race*

Retrieve the name of the race with the lowest mean. You should calculate this value programmatically.

```
var: fastest_race
```

 - the name of the race with the lowest mean.

4 Removing NA rows

Consider the following matrix:

Remove the rows that contain any NAs. Do this programmatically (don't just use integers indices to remove the NA rows - your code should work even if the matrix changes).

Hint: the `any()` and `is.na()` functions will be useful here. Look them up if you're not sure how to use them.

```
var: mat_no_na
```

5 Finding outliers

In the previous homework we wrote a function to identify outliers where outliers were defined as any observations that were more than `coef` standard deviations from the mean.

5.1 *Part 1*

First, write a function called `n_outliers()`. It should be very similar to `find_outliers()` from the previous homework, but instead of returning the values of the outliers, it should return the *number* of outliers in the vector.

var: `n_outliers`- function with two parameters:

- `vals`: a numeric vector
- `coef`: a number - numbers that are more than `coef` standard deviations from the mean will be considered outliers. Default value should be 1.5.

5.2 *Part 2*

Suppose we have data recorded on three variables and 20 observational units (rows are the observational units, columns are the variables). The following code should be used to generate the data:

```
set.seed(212)
n <- 20
n_vars <- 3
rand_mat <- replicate(n_vars, runif(n))
colnames(rand_mat) <- c("var1", "var2", "var3")
```

Use `apply()` to find the number of outliers in each variable, where an outlier is defined as an observation that is more than 1.5 standard deviations from the mean.

var: `rand_mat_n_outliers`